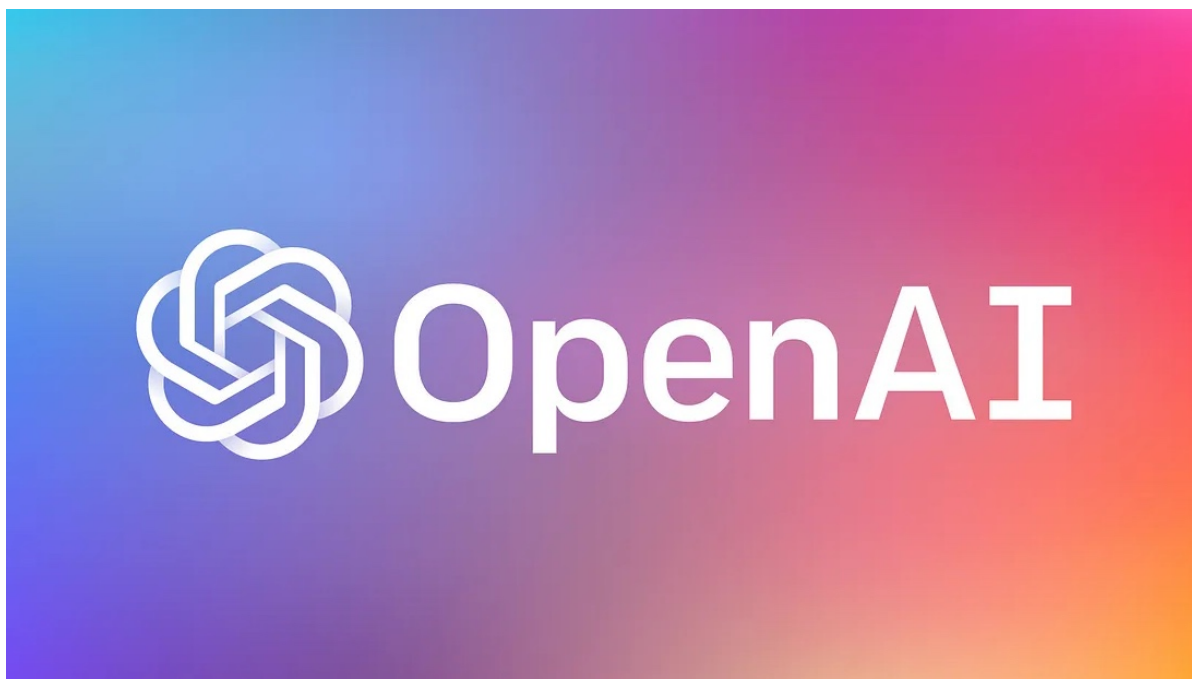


8个使用Codex 节省 Token的方法 (附可复用提示词)

编写 BY @单雨琦 Yuqi Shan

全网同名 小红书 抖音 公众号 视频号 知乎 X

让AI为你所用~



很多人刚开始用 Codex，第一反应都是去压缩提示词。事实上，这点节省通常很有限。

省 Token 最核心的原则是减少无关信息和重复内容。

我把 OpenAI 官方说明翻了一遍，按这个标准，把常见技巧分成了三档。每个关键技巧都提供你可以直接拿去用的提示词。

第一档 真能省，而且越用越值

1. 给长期项目留一份交接文档

一个任务连续做了几十轮以后，上下文会过长，Codex 得重新辨认现在做到哪一步，哪些文件已经改过，哪个报错试过，哪个方案被放弃。旧对话越长，它要翻的东西就越多，使用的 Token 越多。

可以在一个阶段结束时，让 Codex 创建或更新 `PROJECT_STATUS.md`。

可复用提示词：“当前阶段即将结束。请创建或更新 `PROJECT_STATUS.md`，把本次工作沉淀成一份可以继续执行的项目交接文档。

文档需要包含：当前目标、已完成内容、关键技术决策、修改过的核心文件、测试与验证结果、已知问题、尝试过但失败的方案，以及下一步开发顺序。

请结合当前代码、Git 变更和测试结果核对内容。只保留后续开发真正需要的信息，不记录无关讨论，不猜测未确认的结论，也不要写入密码、密钥等敏感信息。”

当前阶段即将结束。请创建或更新PROJECT_STATUS.md，把本次工作沉淀成一份可以继续执行的项目交接文档。

文档需要包含：当前目标、已完成内容、关键技术决策、修改过的核心文件、测试与验证结果、已知问题、尝试过但失败的方案，以及下一步开发顺序。
请结合当前代码、Git 变更和测试结果核对内容。只保留后续开发真正需要的信息，不记录无关讨论，不猜测未确认的结论，也不要写入密码、密钥等敏感信息。

已处理 2m 16s >

已创建并核对项目交接文档：

📄 PROJECT_STATUS.md

下一阶段可以新开一个对话框，让 Codex 先读这份交接文档，再继续开发。

它省下的也不只是 Token，模型换了，AI Agent 换了，隔几天再回来，项目仍然接得上。

2. 把 AGENTS.md 写短，并按目录分层

AGENTS.md 很容易越写越长，今天加一条测试要求，明天补一段代码风格，过几天又把前端、后端和发布流程全塞进去。Codex 最后只改一个按钮，也要带着整份说明开工。

Codex 的规则加载本来就支持分层，它会从项目根目录一路读到当前工作目录，离当前目录更近的规则优先级更高。根目录适合放全局约束，前端、后端和测试目录各自保存专项规则。

一个常见结构可以这样安排。

```
/AGENTS.md
/frontend/AGENTS.md
/backend/AGENTS.md
```

根目录只写整个项目都要遵守的内容。前端组件规范放进 frontend/AGENTS.md，后端测试命令放进 backend/AGENTS.md。Codex 进入对应目录工作时，才需要带上那部分规则。

提示词：“请简化当前的 AGENTS.md，而且按目录分层，只把规则放在真正需要的目录范围内，避免一份超长全局规则覆盖整个仓库。”

这一条对你特别重要。如果你现在有很长的全局 Codex 自定义指令，可以直接用上面的提示词修改。

3. 用 RTK 做压缩

RTK 是一个第三方开源命令行代理，会先过滤、压缩这些输出，再把结果交给 Agent。

开发任务里有一类内容很占地方：命令输出。

git diff、测试日志、目录树和重复报错对人类阅读很方便，交给模型时却经常夹着大量低价值文本。

已处理 1m 27s >

结论：你的全局规则可以从当前约 3263 字节（包含 RTK.md）压缩到约 1000-1400 字节，预计减少约 55%-70% 的固定上下文。主要优化点有三个：

- 合并“简单、MVP、成熟方案、少询问”等相近规则。
- 把“任务归档”移到 Documents/Codex/AGENTS.md。
- 删除单独的 RTK.md 引用，用一行表达核心规则。

官方文档确认，Codex 会依次合并全局、项目和子目录规则，靠近当前目录的规则优先。因此，专用规则放到对应目录最合适。🔗 [OpenAI AGENTS.md 文档](#)

RTK 项目方给出的数据是，常见开发命令可以减少约 60% 到 90% 的输出 Token。这个区间来自项目方测试，具体能省多少，要看命令类型、项目规模和输出内容。日志越长、重复越多，效果通常越明显。

提示词：“帮我安装 github.com/rtk-ai/rtk”

这个操作只需要做一次就可以，系统会自动执行。

第二档 平时使用 Codex 的好习惯

4. 一个阶段结束，就换一个对话框

长对话会积累旧日志、旧方案和已经解决的错误。继续在同一个人任务里做完全不同的工作，Codex 仍要处理这些历史。

里程碑已经完成，接下来要换模块，或者交接文档已经写好，就适合新开任务。

OpenAI 的 Responses API 也提供上下文压缩能力，用来保留后续需要的状态并降低长对话的上下文占用。普通 Codex 用户不必研究接口细节，记住工作习惯就够了。阶段结束时整理状态，下一阶段从干净的上下文开始。

5. 同一条规则只写一次

有的规则是冗余重复的，比如：

完成后检查结果
所有修改都必须测试
结束前一定要验证

压缩成一句“完成修改后运行测试”就够了。

github.com/rtk-ai/rtk 帮我安装

已处理 2m 12s >

已安装并配置完成。

- RTK 版本: `0.45.0`
- 安装方式: Homebrew (官方推荐)
- 已启用 Codex 全局集成
- 已保留原有 `AGENTS.md`, 仅追加 RTK 规则引用
- 功能测试通过: `rtk ls` 正常运行

OpenAI 当前的模型指南也建议把每条指令写一次。重复规则会增加上下文，还可能让 Agent 过度关注流程，反复确认本来可以直接完成的安全操作。

检查范围包括全局说明、项目 `AGENTS.md`、Skill 和任务提示词。相同要求放在一个合适的位置，其他地方删掉重复版本。

6. 提示词写清目标、边界和验收条件

Codex 能自己安排大部分执行步骤。日常任务里，用户更需要讲清三件事，想得到什么，哪些地方不能动，做到什么算完成。

下面这段已经够用。

修复登录页在手机上的横向溢出。
只修改 frontend/。
保持现有视觉风格。
完成后运行相关测试并检查移动端布局。

文件范围、视觉约束和验收方式都保留了，返工风险会低很多。

第三档 省 Token 效果有限的操作

7. 别重复要求 Codex 读取 AGENTS.md

官方文档说明 Codex 会在开始工作前读取适用范围内的 `AGENTS.md`。每条提示词都再加一句“请先认真阅读 `AGENTS.md`”，通常没有额外收益。

这次任务若有额外材料，直接点名文件就行。默认机制已经负责的事情，不必再人工提醒一遍。

8. 提示词字数影响通常没有那么大

提示词简洁当然有好处，关键约束更容易被看见。为了省几十个 Token 删除文件范围、风险边界和验收条件，账往往算不过来。

Codex 一旦理解错任务，可能重新扫描仓库、改错文件、跑一轮无用测试，随后再返工。前面省掉的几十个字，很快就花回去了。

应当优先删掉重复、过期和低信息密度的内容。关键代码、已经确认的决定和测试结果要保留。Codex 看得太少，同样会做错。

我现在会怎么搭配

总结一下，目前我使用的节省 Token 的三个主要操作：

交接任务、修改 AGENTS.md、RTK。

省 Token 这件事，最后拼的是上下文质量。让 Codex 每一轮都拿到足够、准确、与当前决定有关的信息，比抠掉提示词里的几个字有用得多。

谢谢你看到这里啦，如果觉得不错，随手点个赞、评论、关注三连~这对我会是非常大的帮助~我是Yuqi, 以后我也会分享更多这样实用的内容！